PyTorch Fundamentals

Structured Notes

Tensors, GPU Acceleration, Tensor Operations,
Autograd, and Computational Graphs

Contents

1 Introduction to PyTorch **2**
1.1 PyTorch vs TensorFlow . . . 2

2 Tensors: The Fundamental Data Structure **2**
2.1 What Makes PyTorch Tensors Special . . . 2
2.2 Limitations of NumPy for Deep Learning . . . 2

3 Three Critical Tensor Properties **3**
3.1 Property 1: Shape . . . 3
3.2 Property 2: Data Type (dtype) . . . 3
3.3 Property 3: Device . . . 3

4 Tensor Operations **4**
4.1 Element-wise Operations . . . 4
4.2 Dot Product . . . 4
4.3 Matrix-Vector Multiplication . . . 4
4.4 Broadcasting . . . 4

5 CPU vs GPU Execution **5**
5.1 Device Management in PyTorch . . . 5
5.2 Google Colab vs Kaggle . . . 5

6 Autograd: Automatic Differentiation **5**
6.1 The Problem with Manual Gradients . . . 5
6.2 What is Autograd . . . 6
6.3 The requires_grad Flag . . . 6
6.4 Forward and Backward Pass . . . 6
6.5 Important Rules for Gradients . . . 7

7 Computational Graph **7**
7.1 What is a Computational Graph . . . 7
7.2 Dynamic Graph Construction . . . 7
7.3 Training Iterations and Epochs . . . 8

8 Complete Code Reference **8**

9 Summary of Key Concepts **9**

1 Introduction to PyTorch

PyTorch is an open-source **deep learning library** (not a machine learning library) used for building neural networks and deep neural networks. It provides a Pythonic interface that allows developers to see how neural networks are constructed from scratch.

1.1 PyTorch vs TensorFlow

Aspect	PyTorch	TensorFlow
Style	Pythonic, transparent	Less Pythonic
GPU on Windows	Simple with NVIDIA CUDA	Requires WSL or Linux
Keras dependency	Not needed	Required separately
Neural network visibility	Full from-scratch view	Abstracted away
Platform support	Windows, Linux, Mac	Best on Linux

Key Takeaway: PyTorch is preferred for deep learning because of its simplicity in GPU acceleration, Pythonic design, and the ability to build neural networks from scratch without additional libraries like Keras. Linux is the preferred OS for training purposes.

2 Tensors: The Fundamental Data Structure

A **tensor** is PyTorch's fundamental data structure for representing numerical data in deep learning. Deep learning models operate on large collections of numbers, and tensors provide the efficient structure to handle them.

Tensor Hierarchy:

- **Scalar** — 0-dimensional tensor (single number)
- **Vector** — 1-dimensional tensor (array of numbers)
- **Matrix** — 2-dimensional tensor (2D grid of numbers)
- **Higher-order tensor** — 3+ dimensional tensor

2.1 What Makes PyTorch Tensors Special

The distinguishing feature of PyTorch tensors is not their shape (which is identical to NumPy arrays) but their **behavior**:

PyTorch Tensor Capabilities vs NumPy Arrays:

Feature	NumPy	PyTorch
Store numerical data	✓	✓
Vectorized math operations	✓	✓
Track computation history for gradients	×	✓
Move between CPU and GPU seamlessly	×	✓

- **NumPy focus:** Numerical values only
- **PyTorch focus:** Numerical values + computation history

2.2 Limitations of NumPy for Deep Learning

While NumPy is excellent for numerical computation, it has two major limitations:

1. **No gradient tracking:** NumPy does not track gradients. Gradient computation must be done manually using numerical methods — you cannot directly compute $\frac{dy}{dx}$.
2. **No native GPU support:** NumPy operates on CPU only.

Neural networks *can* be built with NumPy, but it requires manually coding gradient computation and storage for each layer, which is extremely time-consuming and error-prone.

3 Three Critical Tensor Properties

Every tensor in PyTorch has three properties that directly affect model behavior:

3.1 Property 1: Shape

Shape determines how tensors interact in matrix operations.

Key Rules:

- For matrix multiplication, the columns of the first matrix must equal the rows of the second.
- A tensor of shape $m \times n$ can only be multiplied with a tensor of shape $n \times p$.
- Most runtime errors in deep learning are shape-related.

Neural network layers are designed with compatible shapes to minimize shape mismatch errors.

3.2 Property 2: Data Type (dtype)

Data type determines numerical precision (e.g., `float32`, `float64`).

Why floating point is mandatory in deep learning:

Gradients require **continuous values**. Integer (discrete) values do not support gradient computation because gradients rely on **continuity**. This is why all deep learning tensors are defined in floating point.

$$\text{Gradient computation} \implies \text{Continuous values} \implies \text{Floating point data type}$$

Note on dtype differences:

- Tensors created directly with `torch.tensor()` default to `float32`.
- Tensors created from NumPy arrays default to `float64` (because NumPy defaults to 64-bit).

This is purely about memory representation (32 bits vs 64 bits), not related to GPU.

3.3 Property 3: Device

Device specifies where computation occurs — CPU or GPU.

Key Rules:

- PyTorch allows the same code to run on different hardware (Windows, Linux, Mac).
- Data must reside on the **same device** as the computation.
- PyTorch does **not** automatically move tensors between devices.
- If not specified, PyTorch defaults to CPU.

- Use `.to(device)` to move tensors from CPU to GPU.

Device keywords:

- NVIDIA GPUs: `"cuda"` (indexed as `cuda:0`, `cuda:1`, etc.)
- Apple Silicon: `"mps"` (Metal Performance Shaders)
- CPU: `"cpu"`

4 Tensor Operations

In deep learning, from neural networks to transformers and LLMs, every computation reduces to four fundamental operations:

4.1 Element-wise Operations

Operations applied independently to each element of a tensor.

4.2 Dot Product

The dot product of two vectors produces a **scalar** value:

$$\mathbf{a} \cdot \mathbf{b} = \sum_i a_i \cdot b_i$$

Example: $\mathbf{a} = [1, 2, 3]$, $\mathbf{b} = [4, 5, 6]$

$$\mathbf{a} \cdot \mathbf{b} = (1)(4) + (2)(5) + (3)(6) = 4 + 10 + 18 = 32$$

In PyTorch: `torch.dot(a, b)`

4.3 Matrix-Vector Multiplication

Matrix-vector multiplication produces a **vector**:

Example:

$$M = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$M \cdot \mathbf{v} = \begin{bmatrix} 1(1) + 2(1) \\ 3(1) + 4(1) \end{bmatrix} = \begin{bmatrix} 3 \\ 7 \end{bmatrix}$$

In PyTorch: `torch.mv(m, v)`

Key distinction: Dot product always gives a scalar. Vector/matrix product gives a vector or matrix.

4.4 Broadcasting

Broadcasting allows operations on tensors of different shapes without explicit loops.

Benefits:

1. **Computational efficiency:** No loop overhead.
2. **Memory efficiency:** No need to replicate data.

3. **Parallel execution:** Enables GPU parallel processing.

Broadcasting replaces explicit loops, making operations faster and more memory-efficient while enabling direct use of GPU parallel processing.

5 CPU vs GPU Execution

Modern deep learning relies heavily on **GPUs** because neural networks involve massive parallel computations. GPU is the first and most important requirement for deep learning.

Why GPUs are essential:

- **Parallel processing:** GPUs can process thousands of operations simultaneously.
- **Optimized for tensors:** GPUs are optimized for matrix and tensor operations.
- **Not just for images:** GPU acceleration is needed for **all** deep learning operations, not just image processing.

5.1 Device Management in PyTorch

Listing 1: Checking and Setting Device

```
import torch

# Check CUDA availability
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Computation device: {device}")

# Move tensor to GPU
x = torch.tensor([1.0, 2.0, 3.0])
x = x.to(device)
print(f"Tensor x is on: {x.device}")  # cuda:0

# Check GPU name
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
```

GPU Indexing: `cuda:0` means the first GPU. Multiple GPUs are indexed as `cuda:0`, `cuda:1`, `cuda:2`, etc.

5.2 Google Colab vs Kaggle

Feature	Google Colab	Kaggle
Storage	20 GB (Google Drive)	~100–300 GB
GPU/TPU	Yes (limited)	Yes (similar)
Large datasets	Impractical	Better suited
Interface	Jupyter-like	Jupyter-like

For large datasets (>20 GB), Kaggle is recommended over Google Colab.

6 Autograd: Automatic Differentiation

6.1 The Problem with Manual Gradients

In classical machine learning (e.g., linear regression), gradients are derived **analytically** using mathematical formulas like the pseudo-inverse for least square optimization. This is manageable for simple models.

Challenges with deep networks:

1. **Error prone:** In a deep neural network with 20–30 layers, manually tracking differentiation at every layer is extremely difficult.
2. **Difficult to scale:** The chain rule grows with network depth — the chain keeps increasing as you go deeper.
3. **Hard to debug:** Finding gradient computation errors in complex networks is very challenging.

Deep learning requires computing derivatives of a loss function with respect to potentially **millions or billions** of parameters.

6.2 What is Autograd

Autograd is PyTorch's engine for computing derivatives **automatically**. Instead of computing gradients symbolically, PyTorch:

1. **Records** all operations performed on tensors.
2. **Builds** a computational graph during the forward pass.
3. **Applies** the chain rule automatically during the backward pass.

This process is called **reverse mode automatic differentiation**.

6.3 The requires_grad Flag

By default, PyTorch does not track gradients. To include a tensor in gradient computation:

```
x = torch.tensor(2.0, requires_grad=True)
```

Key points:

- Set `requires_grad=True` when converting data to tensors for neural network input.
- Without it, the model still works, but gradient values inside layers cannot be inspected.
- Every operation on a tensor with `requires_grad=True` is tracked in the computational graph.

6.4 Forward and Backward Pass

Scalar Example:

Given $y = x^2 + 3x + 1$ with $x = 2.0$:

Forward pass:

$$y = (2)^2 + 3(2) + 1 = 4 + 6 + 1 = 11$$

Backward pass (calling `y.backward()`):

$$\frac{dy}{dx} = 2x + 3 = 2(2) + 3 = 7$$

```
x = torch.tensor(2.0, requires_grad=True)
```

```
y = x**2 + 3*x + 1
print(y.item())     # 11.0
y.backward()
print(x.grad)       # 7.0
```

Vector Example:

Given $z = \sum w_i^2$ with $\mathbf{w} = [1, 2, 3]$:

Forward pass:

$$z = 1^2 + 2^2 + 3^2 = 1 + 4 + 9 = 14$$

Backward pass:

$$\frac{\partial z}{\partial w_i} = 2w_i \implies \nabla z = [2, 4, 6]$$

```
w = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
z = torch.sum(w**2)
z.backward()
print(w.grad)     # tensor([2., 4., 6.])
```

6.5 Important Rules for Gradients

Critical Rules:

1. **Gradient is called on the input tensor**, not the output.
   - `x.grad` works (input tensor).
   - `y.grad` returns `None` with a warning (output tensor).
2. **Without `.backward()`, gradients are `None`.** The backward pass must be explicitly triggered.
3. **`.item()` works only on scalars.** For vector tensors, use `.data` or print directly.
4. **Gradients accumulate by default.** Use `x.grad.zero_()` to reset gradients before the next computation.

7 Computational Graph

7.1 What is a Computational Graph

A **computational graph** is a **Directed Acyclic Graph (DAG)** that records all tensor operations during the forward pass.

Components:

- **Nodes:** Represent tensor operations (e.g., addition, multiplication, activation functions).
- **Edges:** Represent data dependencies between operations. Edges can carry weights.

This is similar to a decision tree structure — every tree is a graph, but every graph is not a tree.

7.2 Dynamic Graph Construction

PyTorch builds the computational graph **dynamically** during execution:

1. **Forward pass:** As operations execute, the graph is built simultaneously.

2. **Backward pass:** The chain rule is applied automatically using the recorded graph.
3. **Weight update:** Parameters are updated based on computed gradients.
4. **Next iteration:** A new graph is built (may differ from the previous one).

The graph can change from iteration to iteration. After each epoch:

- Forward pass forms the graph.
- Back propagation computes gradients.
- Weights are updated.
- In the next iteration, graph nodes reflect updated weights.

This dynamic nature is one of PyTorch's key strengths compared to static graph frameworks.

7.3 Training Iterations and Epochs

The number of iterations (epochs) depends on the data and is determined by monitoring:

- **Loss curve:** Loss should decrease over epochs.
- **Accuracy curve:** Accuracy should increase over epochs.
- **Convergence:** When both curves plateau, training can stop.

Epochs can range from 10 to thousands, depending on the dataset and model complexity.

8 Complete Code Reference

Listing 2: Complete PyTorch Fundamentals Code

```
import torch
import numpy as np

# ---- Tensor Creation ----
# Direct tensor creation (float32 default)
x = torch.tensor([1.0, 2.0, 3.0])
print(f"Shape: {x.shape}")     # torch.Size([3])
print(f"Dtype: {x.dtype}")     # torch.float32
print(f"Device: {x.device}")   # cpu

# From NumPy array (float64 default)
np_arr = np.array([4.0, 5.0, 6.0])
y = torch.tensor(np_arr)
print(f"Dtype: {y.dtype}")     # torch.float64

# ---- Tensor Operations ----
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])

# Dot product (scalar result)
dot = torch.dot(a, b)          # tensor(32.)

# Matrix-vector product (vector result)
m = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
v = torch.tensor([1.0, 1.0])
mv = torch.mv(m, v)            # tensor([3., 7.])

# ---- Device Management ----
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
x_gpu = x.to(device)
print(f"Device: {x_gpu.device}")

# ---- Autograd: Scalar Example ----
x = torch.tensor(2.0, requires_grad=True)
y = x**2 + 3*x + 1            # y = 11
y.backward()
print(f"dy/dx = {x.grad}")     # 7.0

# ---- Autograd: Vector Example ----
x.grad.zero_()                 # Reset gradients
w = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
z = torch.sum(w**2)            # z = 14
z.backward()
print(f"dz/dw = {w.grad}")     # [2., 4., 6.]
```

9 Summary of Key Concepts

Concept	Key Point
PyTorch	Deep learning library, Pythonic, GPU-friendly
Tensor	Fundamental data structure with shape, dtype, device
Shape	Determines valid matrix operations; most errors are shape-related
Data Type	Must be floating point for gradient computation
Device	CPU or GPU; explicit transfer required via `.to(device)`
Broadcasting	Operations on different shapes without loops
Autograd	Automatic differentiation engine
`requires_grad`	Must be True to track gradients
`.backward()`	Triggers backward pass (gradient computation)
`.grad`	Accessed on input tensors, not outputs
Computational Graph	DAG built dynamically during forward pass